

БИЛЕТ 21: Словари (dict), множества (set) и Хеширование

Что такое хеширование и почему это важно?

Хеширование — это функция, которая преобразует данные (например, строку, число, кортеж) в целое число фиксированного размера — хеш-код. Она должна быть детерминированной (для одного и того же ключа всегда возвращать один и тот же хеш) и обеспечивать эффективное распределение, чтобы минимизировать совпадения (коллизии).

В Python для вычисления хеш-значения используется встроенная функция `hash()`

```
print(hash("hello"))    # например: -891827364582734
print(hash(42))          # 42 (число хешируется само в себя)
print(hash((1, 2, 3)))  # 529344067295497451 (кортеж можно хешировать)
```

Свойства хеширования:

1. Если `a == b`, то `hash(a) == hash(b)` (одинаковые объекты → одинаковый хеш)
2. Обратное неверно: одинаковый хеш не гарантирует равенство (коллизии)
3. Хеш вычисляется быстро

Какие объекты можно хешировать (могут быть ключами словаря)?

- ☒ Числа, строки, кортежи (если внутри тоже хешируемые)
- ☐ Списки, словари, множества (изменяемые типы)

Правильно

```
d = {1: "a", "key": "b", (1, 2): "c"} # ☒
```

Неправильно

```
d = {[1, 2]: "value"} # ☐ TypeError: unhashable type: 'list'
```

Словари (dict)

Словарь — это неупорядоченная коллекция пар "ключ → значение". Внутри реализован через **хеш-таблицу**.

Особенности:

- Ключи должны быть **хешируемыми** (неизменяемыми объектами), имеющими методы `__hash__()` и `__eq__()`
- Значения могут быть любого типа данных.
- Словари сохраняют порядок вставки ключей

Создание словарей

Способ 1: фигурные скобки

```
d1 = {"name": "Alice", "age": 25}
```

Способ 2: конструктор dict()

```
d2 = dict(name="Alice", age=25) # ключи без кавычек
```

Способ 3: из списка пар

```
d3 = dict([("name", "Alice"), ("age", 25)])
```

Пустой словарь

```
empty = {} # или dict()
```

Основные операции

```
d = {"a": 1, "b": 2, "c": 3}
```

Доступ к значениям

```
print(d.get("d", 0)) # 0 (безопасно, возвращает default)
```

```
print(d.get("b", 0)) # 2 (ключ есть, default игнорируется)
```

Добавление/изменение

```
d["d"] = 4 # добавляем новую пару
```

```
d["a"] = 100 # изменяем существующую
```

Удаление

```
del d["b"] # удаляет ключ "b"
```

```
value = d.pop("c") # удаляет и возвращает значение (3)
```

Проверка наличия ключа

```
if "a" in d:
    print("Есть!")
```

Длина словаря

```
print(len(d)) # количество пар
```

Важные методы

python

```
d = {"x": 1, "y": 2, "z": 3}
```

Получение всех ключей, значений, пар

```
keys = d.keys()          # dict_keys(['x', 'y', 'z'])
```

```
values = d.values()       # dict_values([1, 2, 3])
```

```
items = d.items()         # dict_items([('x',1), ('y',2), ('z',3)])
```

Итерация по словарю

```
for key in d:
    print(key, d[key])
```

```
for key, value in d.items():
    print(f"{key}: {value}")
```

Объединение словарей

```
d1 = {"a": 1}
```

```
d2 = {"b": 2}
```

```
d1.update(d2)           # d1 становится {"a":1, "b":2}
```

```
d3 = {**d1, **d2}       # новый словарь (Python 3.5+)
```

Очистка

```
d.clear()
```

Множества (set)

Множество — это неупорядоченная коллекция **уникальных** элементов. Тоже основано на хеш-таблице.

Создание множеств

Способ 1: фигурные скобки (не путать со словарем!)

```
s1 = {1, 2, 3, 3, 3}    # {1, 2, 3} (дубликаты удалены)
```

Способ 2: конструктор set()

```
s2 = set([1, 2, 2, 3, 4]) # {1, 2, 3, 4}
```

```
s3 = set("hello")        # {'h', 'e', 'l', 'o'}
```

Пустое множество (важно!)

```
empty = set()           # {} - это пустой словарь!
```

Основные операции с множествами

```
s = {1, 2, 3}
```

Добавление

```
s.add(4)          # {1, 2, 3, 4}
```

```
s.add(2)          # {1, 2, 3, 4} (ничего не меняется)
```

Удаление

```
s.remove(3)       # удаляет 3, если нет → KeyError
```

```
s.discard(5)      # удаляет 5, если нет → ничего не делает
```

```
elem = s.pop()    # удаляет и возвращает произвольный элемент
```

Проверка

```
if 2 in s:
    print("Есть!")
```

Длина

```
len(s)
```

Особенности:

- Элементы должны быть **хешируемыми** (неизменяемыми объектами).
- Дубликаты автоматически удаляются.
- Нет индексов, порядок элементов не сохраняется.

Операции над множествами (как в математике)

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

Объединение (union)

```
print(A | B)      # {1, 2, 3, 4, 5, 6}
```

```
print(A.union(B))
```

Пересечение (intersection)

```
print(A & B)      # {3, 4}
```

```
print(A.intersection(B))
```

Разность (difference)

```
print(A - B)      # {1, 2} (что в A, но не в B)
```

```
print(A.difference(B))
```

Симметрическая разность (xor)

```
print(A ^ B)          # {1, 2, 5, 6} (в одном, но не в обоих)
```

```
# Подмножество/надмножество
```

```
print({1, 2} <= A)    # True (подмножество)
```

```
print(A >= {1, 2})    # True (надмножество)
```

БИЛЕТ 22: Подпрограммы и функции

Понятия

Подпрограмма — это именованный блок кода, который можно вызывать многократно. В Python подпрограммы реализованы как **функции**.

Функция — это подпрограмма, которая выполняет действия и возвращает результат (число, строку символов и др.).

Процедура — это подпрограмма, которая выполняет действия, но не возвращает значения в основную программу.

Зачем нужны функции:

1. **DRY** (Don't Repeat Yourself) — избегаем дублирования кода
2. **Модульность** — разбиваем сложную задачу на части
3. **Повторное использование** — можно использовать в разных программах
4. **Тестирование** — легче проверять отдельные куски

Определение функции (def)

```
def имя_функции(параметр1, параметр2, ...):  
    """Документация (docstring) — описание что делает функция"""  
    # Тело функции  
    результат = параметр1 + параметр2  
    return результат
```

Правила именования:

- Использовать `snake_case`: `calculate_average`, `get_user_name`
- Имя должно отражать действие (глагол): `get_`, `set_`, `calc_`, `find_`

Если оператор `return` не указан, функция возвращает `None`. Функция может возвращать сразу несколько значений — в этом случае они автоматически упаковываются в кортеж.

Вызов функции осуществляется по её имени с передачей аргументов в круглых скобках.

Типы функций

- **Встроенные функции** — предоставляются языком и доступны сразу после запуска интерпретатора (например, `print`, `int`, `input`, `min`, `max`, `range`).
- **Пользовательские функции** — создаются программистом для решения конкретных задач.
- **Лямбда-функции (анонимные функции)** — определяются с помощью ключевого слова `lambda`, могут содержать лишь одно выражение. Используются там, где важна сама операция, а не повторное обращение к функции по имени.
- **Функции высшего порядка** — принимают другие функции в качестве аргументов или возвращают их в качестве результата (например, `map()`, `filter()`, `reduce()`).
- **Функции-генераторы** — работают похоже на обычные, но вместо `return` используют `yield`, отдавая результат по одному элементу за раз.
- **Рекурсивные функции** — вызывают сами себя.

Оператор return

`return` выполняет ДВЕ важные функции:

1. **Завершает выполнение функции** (дальнейший код не выполняется)
2. **Возвращает значение** вызывающему коду

Особенности return

```
def func1():  
    return # возвращает None  
  
def func2():  
    pass # тоже возвращает None (неявно)
```

```
result1 = func1() # None  
result2 = func2() # None
```

Множественный return:

```
def divide(a, b):  
    if b == 0:  
        return None, "Ошибка: деление на ноль"  
    return a / b, "Успешно"  
  
value, message = divide(10, 2)
```

```
print(value, message) # 5.0 Успешно
```

Жизненный цикл функции

1. Определение (создание функции в памяти)

```
def greet(name):  
    return f"Hello, {name}!"
```

2. Вызов (создание контекста выполнения)

```
message = greet("Alice") # Передача управления в функцию
```

3. Выполнение тела функции

4. Возврат (return) и уничтожение локальных переменных

5. Продолжение выполнения основного кода

```
print(message)
```

БИЛЕТ 23: Аргументы функций

Аргументы функций в Python — это значения, которые передаются в функцию при её вызове и становятся переменными внутри тела функции. Они позволяют параметризовать работу функции и сделать её универсальной.

Виды аргументов

1. Позиционные аргументы. Передаются в том порядке, в котором указаны параметры при объявлении функции. Порядок передачи важен: первый аргумент идёт в первый параметр, второй — во второй и т. д. Пример:

```
def calculate_price(price, quantity):  
    total = price * quantity  
  
    print(f"Цена за {quantity} единиц товара: {total} руб.")
```

```
calculate_price(100, 2) # Вывод: Цена за 2 единицы товара: 200 руб.
```

2. Именованные (ключевые) аргументы. Передаются с явным указанием имени параметра, что позволяет не соблюдать порядок, указанный в определении функции. Это повышает читаемость, особенно когда параметров много.

```
calculate_price(price=100, quantity=2) # Порядок не важен
```

3. Аргументы по умолчанию. Для параметров можно задать значения по умолчанию при определении функции. Если при вызове функции этот аргумент не указан, используется значение по умолчанию.

```
def say_hello(name="Tom"):
    print(f"Hello, {name}")

say_hello() # name будет иметь значение "Tom"
say_hello("Bob") # name будет равен "Bob"
```

4. Аргументы переменной длины (*args и kwargs). Используются, когда заранее неизвестно, сколько аргументов нужно передать.

- ***args** — собирает все позиционные аргументы в кортеж.
- ****kwargs** — собирает все именованные аргументы в словарь, где ключи — имена параметров, а значения — переданные аргументы.

```
def show_info(*args, **kwargs):
    print("Позиционные аргументы (*args):")
    for arg in args:
        print(arg)

    print("Именованные аргументы (**kwargs):")
    for key, value in kwargs.items():
        print(f"{key} = {value}")

show_info("Анна", "Борис", group="ИСТ-21", course=2)
```

БИЛЕТ 24: Область видимости переменных

Область видимости переменных в Python — это часть программы, в которой переменная доступна для использования. Она определяет, где и как переменная может быть применена в коде.

Правило LEGB

- **LEGB** — это аббревиатура, которая обозначает порядок поиска переменных в коде:
- **Local** — локальная область видимости (внутри текущей функции).
- **Enclosing** — вложенная (охватывающая) область видимости (для вложенных функций).
- **Global** — глобальная область видимости (на уровне модуля).
- **Built-in** — встроенная область видимости (предопределённые функции, типы, исключения и другие атрибуты Python). Когда интерпретатор встречает переменную, он ищет её в указанном порядке и прекращает поиск, как только находит.

```
x = 10
def outer():
    x = 20
    def inner():
        x = 30 # Локальная переменная
        print(f"Локальная x: {x}")
```

```
    inner()
    print(f"Enclosing x: {x}")

outer()
```

Основные типы областей видимости

Локальная область видимости (Local). Переменные, определённые внутри функции, доступны только внутри этой функции и существуют только во время её выполнения. После завершения работы функции такие переменные удаляются из памяти.

```
def my_function():
    local_var = 10 # Локальная переменная
    print(local_var)

my_function()
print(local_var) # ✗ NameError: name 'local_var' is not defined
```

Глобальная область видимости (Global). Переменные, объявленные на верхнем уровне модуля (вне всех функций), доступны в любой части программы, включая внутри функций.

```
counter = 0 # Глобальная переменная

def increment():
    global counter # Обязательно для ИЗМЕНЕНИЯ глобальной
    counter += 1

def read_counter():
    # global НЕ нужен для ЧТЕНИЯ
    return counter

increment()
increment()
print(read_counter()) # 2
```

Вложенная (enclosing) область видимости. Существует только для вложенных функций. Переменные из внешней функции доступны во вложенной.

```
def make_counter():
    count = 0 # Переменная внешней функции
    def increment():
        nonlocal count # Без nonlocal будет ошибка
        count += 1
        return count
    def get_count():
        nonlocal count # Тоже нужно для чтения?
        return count # На самом деле для чтения nonlocal не обязателен
    return increment, get_count

inc, get = make_counter()
print(inc()) # 1
```

```
print(inc()) # 2
print(get()) # 2
```

Встроенная область видимости (Built-in). Содержит ключевые слова, функции, исключения и другие атрибуты, встроенные в Python.

БИЛЕТ 25: Рекурсивные функции

Рекурсивные функции в Python — это функции, которые в процессе своей работы вызывают саму себя. Такие функции используются для решения задач, которые могут быть разбиты на более простые подзадачи того же типа.

Принцип работы

Рекурсивная функция должна иметь два компонента:

1. **Базовый случай** — условие, при котором функция перестаёт вызывать саму себя и возвращает конкретное значение. Без базового случая рекурсия продолжалась бы бесконечно.
2. **Рекурсивный случай** — когда функция вызывает сама себя, обычно с изменёнными аргументами. Главное правило: каждый следующий вызов должен приближать к базовому случаю.

Примеры

- **Вычисление факториала числа.** Факториал числа n (обозначается $n!$) — произведение всех натуральных чисел от 1 до n включительно. В Python рекурсивная реализация выглядит так:

```
def factorial(n): # Базовый случай
    if n == 0 or n == 1:
        return 1 # Рекурсивный случай
    else:
        return n * factorial(n-1).
```

- **Подсчёт суммы чисел от 1 до n .** В Python рекурсивная функция для подсчёта суммы чисел от 1 до n :

```
def sum_to_n(n): # Базовый случай
    if n <= 1:
        return n # Рекурсивный случай
    else:
        return n + sum_to_n(n-1).
```

Когда использовать рекурсию

Рекурсия хороша для:

- Древовидных структур (HTML DOM, файловая система, синтаксические деревья)

- Разделяй-и-властвуй алгоритмов (быстрая сортировка, сортировка слиянием)
- Обход графов (DFS)
- Задач с естественным рекурсивным определением (факториал, числа Фибоначчи)

Рекурсия плоха для:

- Глубоких структур (> 1000 уровней) — ошибка RecursionError
- Простых итераций (лучше использовать циклы)
- Когда важна производительность (вызов функции имеет накладные расходы)

БИЛЕТ 26: Модули и пакеты. Импорт. Стандартная библиотека

Модули в Python — это файлы с расширением `.py`, содержащие определения функций, классов, переменных и исполняемый код. Они позволяют структурировать программу, повторно использовать код и повышать читаемость.

Пакет — это директория, содержащая модули и специальный файл `__init__.py`, который указывает Python, что эта директория должна рассматриваться как пакет. С версии Python 3.3 любая папка (даже без `__init__.py`) считается пакетом.

Импорт — это процесс загрузки модуля в память и предоставления доступа к его компонентам.

Основные способы импорта:

- **Импорт всего модуля.** Например, `import mymodule` позволяет использовать функции и константы модуля через префикс `mymodule`
- **Импорт отдельных элементов.** С помощью ключевого слова `from` можно импортировать только нужные функции, классы или переменные из модуля. Например, `from mymodule import add, multiply`
- **Импорт всех элементов из модуля.** Не рекомендуется, так как может привести к конфликтам имён.

При первом импорте модуля Python:

1. Ищет файл модуля (`sys.path`)
2. Компилирует в байт-код (если нужно)
3. Выполняет весь код модуля
4. Создаёт объект модуля в памяти

Стандартная библиотека Python

Стандартная библиотека Python — это набор модулей, которые включены в дистрибутив Python и могут использоваться без дополнительной установки. Она предоставляет готовые решения для распространённых задач программирования.

В стандартную библиотеку входят:

- модули для работы с системой (например, `sys`, `os`);
- инструменты для обработки данных (например, `json`, `csv`);
- утилиты для работы с сетью (например, `urllib`, `http`);
- средства для создания пользовательских интерфейсов и многое другое.

Некоторые примеры модулей стандартной библиотеки:

- `math` — математические функции и константы;
- `random` — генерация случайных чисел;
- `datetime` — работа с датами и временем;
- `os.path` — манипуляции с путями к файлам;
- `json` — работа с JSON-данными.

БИЛЕТ 27: Работа с файлами. Открытие и закрытие. Режимы доступа. Менеджер контекста `with`

Каждая файловая операция проходит 3 этапа:

1. **Открытие** — получение файлового объекта
2. **Чтение/запись** — операции с данными
3. **Закрытие** — освобождение ресурсов

Для открытия файла применяется функция `open()`. Она принимает два обязательных параметра: путь к файлу и режим доступа. Функция возвращает объект файла, с которым можно выполнять операции чтения, записи и другие действия. `open(file, mode, encoding=None)`

Режимы доступа (`mode`)

- `'r'` — чтение (режим по умолчанию). Если файл не существует, возникает ошибка `FileNotFoundError`
- `'w'` — запись. Если файл существует, его содержимое удаляется; если нет — создаётся новый.
- `'a'` — добавление. Данные записываются в конец существующего файла; если файла нет — создаётся новый.
- `'r+'` — чтение и запись. Файл должен существовать; запись идёт поверх существующего текста, начиная с начала файла.
- `'w+'` — чтение и запись. Файл очищается или создаётся заново.

- 'a+' — чтение и добавление. Файл открывается в режиме добавления, но можно также его читать; при записи курсор всегда находится в конце.
- 'b' — бинарный режим. Используется для работы с нетекстовыми файлами (например, изображениями). Применяется вместе с другими режимами (например, 'rb' для чтения бинарного файла).

После завершения работы с файлом его необходимо закрыть, чтобы освободить системные ресурсы. Для этого используется метод `close()`. Однако явный вызов `close()` не всегда гарантирует корректное закрытие, например, при возникновении исключения.

Конструкция `with` — это менеджер контекста, который автоматически закрывает файл после завершения работы с ним, даже если в блоке кода возникло исключение. Это более безопасный и рекомендуемый способ работы с файлами.

Пример:

```
with open('example.txt', 'r', encoding='utf-8') as file:
    content = file.read()
    # обработка данных
# Здесь файл уже закрыт, даже если произошла ошибка
```

БИЛЕТ 28: Чтение и запись файлов. Методы `read()`, `write()`, `readline()`

Метод `read()`

Синтаксис:

`file.read(size)`, где `file` — объект файла, а `size` — опциональный числовой аргумент, указывающий количество считываемых символов или байтов. Если `size` не указан или отрицателен, метод возвращает всё содержимое файла в виде строки (в текстовом режиме) или байтов (в бинарном режиме).

Чтение всего файла: `content = file.read()`

Чтение определённого количества символов: `content = file.read(20)`

Если конец файла достигнут, `read()` возвращает пустую строку ''

Метод `write()`

Синтаксис:

`file.write(string)`, где `file` — объект файла, а `string` — строка для записи. Метод записывает содержимое строки в файл и возвращает количество записанных символов.

Особенности:

- Метод работает только со строками. Если нужно записать числа или другие типы данных, их предварительно нужно преобразовать в строку.

- Символ переноса строки (`\n`) нужно добавлять вручную, если это необходимо.
- При открытии файла в режиме `'w'` содержимое существующего файла удаляется, а новый создаётся.

Пример:

```
with open('output.txt', 'w') as file:
    n = file.write("Это первая строка
А вот и вторая
И третья — последняя
")
    print(n) # Выведет количество записанных символов, включая
пробелы и
```

Метод `readline()`

Синтаксис:

`file.readline()`, где `file` — объект файла. Метод считывает одну строку из файла. В конце строки остаётся символ переноса (`\n`), который может отсутствовать в последней строке, если файл не заканчивается на перенос.

Особенности:

- Метод удобен для последовательной обработки больших файлов, так как не требует загрузки всего файла в память.
- Можно использовать в цикле для построчного чтения: `for line in file: print(line, end='')`.
- Пустая строка представлена как `' \n '` — строка, содержащая только один символ переноса.

Другие методы

- `readlines()` — считывает все строки файла и возвращает их в виде списка, где каждый элемент — строка из файла.
- `writelines()` — записывает элементы списка строк в файл по очереди, без разделителя.

БИЛЕТ 29: Обработка исключений

Обработка исключений в Python — это механизм, который позволяет программе грамотно реагировать на непредвиденные ситуации и продолжать работу, вместо того чтобы аварийно завершаться при первой же ошибке.

Зачем нужна обработка исключений:

- предотвращать аварийное завершение программы;
- отделять код обработки ошибок от основной логики — делает код более читаемым и понятным;

- централизованно обрабатывать разные типы ошибок — упрощает структуру программы;
- делать программу более устойчивой — она сможет корректно работать даже в неожиданных ситуациях.

Виды

В Python есть встроенные исключения, которые обрабатывают большинство типовых ошибок. Некоторые из них:

- **TypeError** — операция или функция применяется к объекту несоответствующего типа.
- **ValueError** — операция или функция получает аргумент неподходящего значения, например, исключение возникает, если попытаться преобразовать строку в число.
- **ZeroDivisionError** — ошибка при делении на ноль.

Синтаксис

Основной механизм обработки исключений в Python — **блок try-except**.

Некоторые особенности структуры блока try-except:

- **try** — блок кода, в котором могут возникнуть исключения. Python пытается выполнить этот код, и если возникает исключение, выполнение блока прерывается.
- **except** — блок кода, который выполняется, когда в try-блоке возникает исключение указанного типа. Можно указать конкретные типы исключений или перехватывать все исключения (последнее не рекомендуется).
- **else** — необязательный блок, который выполняется только если в try-блоке не возникло исключений. Полезен для кода, который должен выполняться при успешном завершении основных операций.
- **finally** — необязательный блок, который выполняется всегда, независимо от того, возникло исключение или нет. Идеален для освобождения ресурсов или завершающих операций.

Пример обработки исключений при работе с файлами:

```
try:
    with open("data.txt", "r") as file:
        content = file.read()
        data = int(content)
        result = 100 / data
except FileNotFoundError:
    print("Файл не найден. Создаю новый файл с значением по умолчанию.")
except ValueError:
```

```
    print("Содержимое файла не является числом. Использую значение  
по умолчанию.")  
except ZeroDivisionError:  
    print("Деление на ноль невозможно. Использую альтернативный  
расчёт.")  
else:  
    print(f"Успешно рассчитан результат: {result}")  
finally:  
    print("Операция завершена, независимо от результата.")
```

БИЛЕТ 30: Библиотеки Python. pip и виртуальные окружения

Библиотеки в Python — это наборы готового кода, который можно использовать в программах. Они позволяют экономить время, избегая написания кода «с нуля», и ускорять достижение результатов за счёт решений, созданных другими разработчиками. В Python существуют встроенные модули (включены в стандартную библиотеку), сторонние библиотеки (устанавливаются дополнительно) и собственные модули, которые создаёт разработчик.

pip — это менеджер пакетов для Python, который упрощает установку, обновление и управление библиотеками. С его помощью можно быстро устанавливать пакеты из центрального источника — Python Package Index (PyPI), а также управлять зависимостями.

Виртуальные окружения — это изолированные среды Python, которые позволяют устанавливать пакеты локально для конкретного проекта, не влияя на глобальную установку Python и другие проекты. Они помогают избежать конфликтов версий библиотек и системных библиотек, а также упрощают перенос проектов между машинами.

Некоторые возможности pip:

- установка пакетов из PyPI и других источников (например, из URL или локальных каталогов);
- обновление и удаление пакетов;
- управление зависимостями;
- создание списка зависимостей проекта для их последующей установки.

Некоторые команды pip:

- `pip install <имя_пакета>` — установка пакета;
- `pip install --upgrade <имя_пакета>` — обновление пакета;
- `pip uninstall <имя_пакета>` — удаление пакета;
- `pip list` — вывод списка установленных пакетов;

- `pip freeze` — создание списка установленных пакетов в формате, который можно использовать для установки.

Для создания виртуального окружения используется модуль `venv`, который встроен в Python (начиная с версии 3.3). **Пример создания:** `python -m venv myenv`

После выполнения команды Python создаст отдельную директорию с собственным интерпретатором, менеджером пакетов и всеми зависимостями проекта.

Активация виртуального окружения зависит от операционной системы: в Windows: `myenv\Scripts\activate`

После активации все команды `python` и `pip` будут выполняться внутри этого окружения.

Преимущества виртуальных окружений:

- изоляция зависимостей между проектами;
- упрощение управления пакетами и их версиями;
- возможность создавать несколько версий Python в одном окружении;
- упрощение переноса проектов на другие машины.

Для выхода из виртуального окружения используется команда `deactivate`.

БИЛЕТ 31: Основные принципы ООП: инкапсуляция, наследование, полиморфизм

Что такое ООП?

Объектно-ориентированное программирование (ООП) — это парадигма программирования, в которой программа строится из **объектов**, имеющих **состояние** (атрибуты) и **поведение** (методы).

Инкапсуляция — это принцип ООП, который заключается в объединении данных и методов, работающих с этими данными, в единый объект и скрытии деталей реализации от внешнего мира. Цель — защитить данные от неконтролируемого изменения и предоставить контролируемый интерфейс для работы с объектом.

В Python инкапсуляция реализуется через соглашения об именовании атрибутов и методов:

- **Публичные** — доступны из любого места программы без префикса.
- **Защищённые** — доступны только в пределах класса и его подклассов, обозначаются одним подчёркиванием (например, `_attribute`).

- **Приватные** — доступны только в пределах класса, обозначаются двумя подчёркиваниями (например, `__attribute`).

Пример:

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self._pages = pages # Приватный атрибут

    def set_pages(self, pages):
        if pages > 0:
            self._pages = pages
```

Наследование — это механизм, позволяющий создавать новые классы на основе существующих. Дочерний класс (подкласс) наследует атрибуты и методы родительского класса (суперкласса), но может расширять или изменять их функциональность.

Преимущества наследования:

- повторное использование кода;
- построение иерархий классов;
- лёгкость добавления новой функциональности.

Пример:

```
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def make_sound(self):
        return "Некоторый звук животного"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Canis familiaris")
        self.breed = breed

    def make_sound(self):
        return "Гав!"
```

Полиморфизм — это способность объектов разных классов реагировать на одинаковые методы или операции по-разному. В переводе с греческого «полиморфизм» означает «много форм».

В Python полиморфизм реализуется несколькими способами:

- **Полиморфизм с функциями и методами** — одна функция работает с разными типами данных (например, встроенная функция `len()`).
- **Полиморфизм с наследованием** — подклассы переопределяют методы родительского класса.
- **«Утиная типизация» (duck typing)** — если объект умеет делать то, что нужно, не важно, какого он типа.
- **Полиморфизм операторов** — операторы (+, -, *) работают по-разному с разными типами.

Пример:

```
class Animal:
    def make_sound(self):
        raise NotImplementedError("Подклассы должны реализовать этот метод")

class Cat(Animal):
    def make_sound(self):
        return "Мяу!"

animals = [Cat(), Dog()]
for animal in animals:
    print(animal.make_sound()) # Каждое животное издаёт свой звук
```

Преимущества полиморфизма:

- упрощение кода;
- гибкость — легко добавлять новые типы, не меняя существующий код;
- читаемость — код становится понятнее и проще для сопровождения.

БИЛЕТ 32: Понятие класса и объекта. Создание класса. `__init__` и `self`

Класс в Python — это шаблон или «чертёж», который определяет структуру и поведение объектов определённого типа. Он описывает, какие данные (атрибуты) будут содержать объекты и какие операции (методы) они смогут выполнять. Класс можно представить как проект дома, а объект — как конкретный дом, построенный по этому проекту.

Объект (экземпляр класса) — это конкретная реализация класса, созданная в памяти. Каждый объект имеет своё уникальное состояние (значения атрибутов) и может выполнять поведение, определённое в классе (методы).

Класс в Python создаётся с использованием ключевого слова `class`. После него указывается имя класса, затем ставится двоеточие. Тело класса оформляется с отступом.

Пример синтаксиса:

```
class MyClass:  
    pass
```

`**__init__` — специальный метод (конструктор), который вызывается автоматически при создании нового объекта. Он используется для инициализации атрибутов объекта.

Синтаксис:

```
def __init__(self, *args, **kwargs):  
    # код инициализации
```

Пример:

```
class Car:  
    def __init__(self, color, consumption, tank_volume, mileage=0):  
        self.color = color  
        self.consumption = consumption  
        self.tank_volume = tank_volume  
        self.reserve = tank_volume  
        self.mileage = mileage  
        self.engine_on = False
```

`**self` — специальный параметр, который ссылается на конкретный экземпляр класса. Он автоматически передаётся при вызове метода через объект. Через `self` можно получить доступ к атрибутам и другим методам экземпляра.

Особенности:

- `self` не является обязательным параметром, но это общепринятое соглашение в Python, которое делает код более читаемым.
- Технически можно использовать любое имя для первого параметра (например, `this`, `me`), но использование `self` упрощает понимание кода.

Пример использования:

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def greet(self):  
        return f"Привет! Меня зовут {self.name}, мне {self.age} лет."
```

```
p1 = Person("Emil", 25)  
print(p1.greet()) # Привет! Меня зовут Emil, мне 25 лет.
```

БИЛЕТ 33: Атрибуты класса и экземпляра. `@classmethod` и `@staticmethod`

Атрибуты класса — это переменные, которые определены на уровне класса и разделяются между всеми его экземплярами. Они объявляются непосредственно внутри класса, но вне любых методов. Например:

```
class Student:
    school = "Школа №1" # Атрибут класса, общий для всех экземпляров

    def __init__(self, name):
        self.name = name # Атрибут экземпляра, уникальный для каждого студента
```

Атрибуты экземпляра — это переменные, которые хранят данные, уникальные для каждого конкретного объекта. Они определяются внутри методов (обычно в `__init__`) и доступны через `self`. Например:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Когда использовать:

- Атрибуты экземпляра — для данных, которые различаются между объектами (имя, возраст и т. д.).
- Атрибуты класса — для констант или настроек по умолчанию для всех экземпляров, данных, которые должны быть общими для всех экземпляров, отслеживания данных, связанных со всем классом (например, количество созданных экземпляров).

`@classmethod` Декоратор

`@classmethod` используется для создания методов, которые работают с классом в целом, а не с конкретным экземпляром. Такие методы принимают в качестве первого параметра ссылку на класс (`cls`), а не на экземпляр (`self`). Это позволяет обращаться к атрибутам класса внутри метода класса, даже если этот атрибут переопределён в подклассе.

Пример использования:

```
class BaseEntity:
    @classmethod
    def get_table(cls):
        return cls._table
```

```
class User(BaseEntity):
    _table = "users"

print(User.get_table()) # 'users'
```

Часто @classmethod используется для альтернативных конструкторов.

Например, с его помощью можно создать объект из класса, используя `cls()`, вместо прямого вызова имени класса.

@staticmethod Декоратор

`@staticmethod` обозначает обычную функцию, определённую внутри класса, которая не привязана к данным конкретного объекта. Статические методы не принимают ни `self`, ни `cls` в качестве первого параметра. Они обычно используются для вспомогательных функций, логически связанных с классом, но не требующих доступа к состоянию класса или экземпляра

Пример:

```
class MathUtils:
    @staticmethod
    def is_prime(number):
        if number < 2:
            return False
        for i in range(2, int(number**0.5) + 1):
            if number % i == 0:
                return False
        return True

print(MathUtils.is_prime(7)) # True
```

Статические методы можно вызывать напрямую от класса, без создания объекта.

БИЛЕТ 34: Инкапсуляция. Соглашения об именах. @property

Инкапсуляция в Python — это принцип объектно-ориентированного программирования (ООП), который заключается в объединении данных и методов, работающих с ними, в единый объект, а также в сокрытии внутренней реализации от внешнего кода. Это позволяет защитить данные от случайного изменения, упростить отладку и поддержку программы.

Соглашения об именовании

В Python нет жёстких модификаторов доступа (как `public`, `private` или `protected`), как в некоторых других языках. Вместо этого используются **соглашения об именовании**.

Префикс	Уровень доступа	Описание
Нет префикса	Публичный (public)	Атрибут или метод доступны отовсюду.
<code>_</code> (один нижний подчёркивание)	Защищённый (protected)	Атрибут или метод предназначены для внутреннего использования в классе или его подклассах, но технически доступны извне. Это скорее предупреждение для разработчика, чем строгий запрет.
<code>__</code> (два нижних подчёркивания)	Приватный (private)	Вызывают искажение имени (name mangling). Python внутренне переименовывает атрибут, добавляя к нему имя класса (например, <code>__private_variable</code> становится <code>_ClassName__private_variable</code>). Это усложняет случайный доступ, но не делает атрибут полностью приватным.

Декоратор `@property`

Декоратор `@property` — это встроенный в Python инструмент, который позволяет определять методы класса как свойства. Это позволяет обращаться к методам как к атрибутам, без скобок, упрощая и улучшая читаемость кода. С помощью `@property` можно создавать **геттеры** (методы для получения значения атрибута) и **сеттеры** (методы для установки значения атрибута с возможностью валидации данных). Также возможно использование **делитеров** (методов для удаления атрибута).

Пример использования:

```
class Circle:
    def __init__(self, radius):
        self._radius = radius # Внутреннее хранение радиуса

    @property
    def radius(self):
        """Геттер для радиуса."""
        return self._radius

    @radius.setter
    def radius(self, value):
        """Сеттер для радиуса с валидацией."""
        if not isinstance(value, (int, float)) or value <= 0:
```

```
        raise ValueError("Радиус должен быть положительным  
числом.")  
        self._radius = value
```

Преимущества использования @property:

- **Инкапсуляция** — скрывает детали реализации и предоставляет контролируемый доступ к данным.
- **Читаемость кода** — обращение к данным выглядит естественно, как будто это обычные переменные.
- **Гибкость** — позволяет изменять внутреннюю логику без изменения интерфейса класса.
- **Валидация данных** — можно проверять значения перед их установкой.
- **Вычисляемые свойства** — свойства могут рассчитываться на лету на основе других атрибутов.

БИЛЕТ 35: Наследование классов. `super()`. Множественное наследование

Наследование в Python — механизм, позволяющий создавать новые классы на основе существующих. Производный класс получает атрибуты и методы базового класса, но может добавлять свои или изменять существующие.

Функция `super()` — встроенный инструмент в Python, который даёт доступ к методам родительского класса через наследование. Она позволяет вызывать методы без явного указания имени родительского класса. Особенно полезна при переопределении методов, например `__init__`, так как помогает избежать дублирования кода и упрощает сопровождение программ.

Множественное наследование — возможность для класса наследоваться от нескольких базовых классов одновременно. В этом случае производный класс получает доступ ко всем атрибутам и методам всех родительских классов.

Порядок разрешения методов (MRO)

При множественном наследовании Python использует **Method Resolution Order (MRO)** — алгоритм для определения порядка поиска методов в иерархии классов. Python применяет алгоритм **СЗ-линеаризации**, который гарантирует, что:

- каждый класс появляется в MRO только один раз;
- сохраняются отношения родитель-потомок;
- вызовы методов остаются предсказуемыми и без конфликтов.

Посмотреть MRO можно с помощью атрибута `__mro__` класса.

Роль `super()` при множественном наследовании

Функция `super()` играет ключевую роль в управлении порядком вызова методов родительских классов при множественном наследовании. Она не просто вызывает метод непосредственного родителя, а обращается к следующему классу в MRO.

Пример:

```
class A:
    def some_method(self):
        print("Method from class A")

class B:
    def some_method(self):
        print("Method from class B")

class C(A, B):
    def some_method(self):
        super().some_method()
        print("Method from class C")

obj = C()
obj.some_method() # Выведет: Method from class A, Method from class C
```

В этом примере `super()` помогает вызвать метод из класса A в MRO, за которым следует его собственный метод.

Важные нюансы использования `super()`

- **Вызов `super()` в конструкторе.** При множественном наследовании важно, чтобы все базовые классы вызывали `super()` в своих методах `__init__`, чтобы цепочка инициализации была полной. Если в переопределённом методе не вызвать `super()`, это может нарушить цепочку инициализации и привести к неправильному состоянию объекта.
- **Соответствие сигнатуры.** При переопределении методов с разными сигнатурами необходимо убедиться, что вызов `super()` соответствует родительскому методу.
- **Конфликты имён.** Если в базовых классах есть совпадающие имена, приоритет получит тот класс, что указан первым в MRO.
- **Ромбовидное наследование.** Это ситуация, когда класс наследуется от нескольких классов, имеющих общего предка. В таком случае может возникнуть проблема множественной инициализации общего класса. `super()` помогает избежать этого, следуя алгоритму MRO.

БИЛЕТ 36: Полиморфизм в Python. Переопределение методов базового класса. Утиная типизация.

Полиморфизм в Python — это принцип объектно-ориентированного программирования, который позволяет объектам разных классов использовать методы с одинаковыми именами, но разной реализацией. Это делает код более гибким и удобным для расширения.

Переопределение методов базового класса

Полиморфизм часто реализуется через **наследование**. Подклассы могут переопределять методы родительского класса, чтобы изменить их поведение. При этом сигнатура методов совпадает, и к ним может обращаться вызывающий код вне зависимости от класса объекта.

Пример:

```
class Animal:
    def speak(self):
        raise NotImplementedError("Каждое животное должно определить свой звук!")

class Dog(Animal):
    def speak(self):
        return "гав!"

class Cat(Animal):
    def speak(self):
        return "мяу!"

dog = Dog()
cat = Cat()

print(dog.speak()) # Гав-гав!
print(cat.speak()) # Мяу-мяу!
```

В этом примере метод `speak()` определён в базовом классе `Animal`, но каждый подкласс реализует свою версию. При вызове `speak()` для экземпляра `Dog` или `Cat` Python вызывает соответствующий метод в зависимости от типа объекта.

Для доступа к методам родительского класса в подклассе можно использовать функцию `super()`.

Утиная типизация — это подход к полиморфизму в Python, основанный на принципе: «Если нечто выглядит как утка, плавает как утка и крикает как утка, то

это, вероятно, и есть утка». В контексте программирования это означает, что важно не то, каким классом является объект, а то, какие методы и свойства он поддерживает.

Преимущества утиной типизации:

- **Гибкость.** Возможность работать с объектами, которые не были специально спроектированы для совместной работы.
- **Слабая связанность.** Объекты не зависят от иерархии классов.
- **Простота расширения.** Легко создавать новые совместимые типы.
- **Меньше избыточного кода.** Не требуется создавать абстрактные базовые классы и иерархии.

Пример утиной типизации:

```
def make_it_swim_and_quack(obj):  
    obj.swim()  
    obj.quack()
```

```
class Duck:  
    def swim(self):  
        return "плывёт"  
  
    def quack(self):  
        return "крякает"
```

```
make_it_swim_and_quack(Duck()) # Крякает
```

Утиная типизация широко применяется во встроенных функциях и протоколах Python. Например, функция `sorted()` работает с любым объектом, поддерживающим итерацию, а контекстные менеджеры могут быть реализованы в любом классе, предоставляющем методы `__enter__()` и `__exit__()`.

В Python 3.8 были добавлены **протоколы**, которые позволяют описывать поведение и реализовывать интерфейсы статически.

БИЛЕТ 37: Абстрактные классы и методы. Модуль `abc`. Назначение и использование.

Абстрактные классы и методы в Python — это инструменты для определения контрактов, которые обязывают подклассы реализовывать определённые методы. Они позволяют стандартизировать интерфейс, обеспечить согласованность реализации и упростить масштабируемость кода.

Модуль `abc` (Abstract Base Classes) — ключевой инструмент для работы с абстрактными классами в Python. Он был введён в Python 2.6 для формализации

абстрактных интерфейсов и с тех пор стал неотъемлемой частью стандартной библиотеки.

Основные компоненты модуля `abc`

- **Класс `abc`.** Базовый класс для создания абстрактных классов. Все определяемые абстрактные классы наследуются от этого класса.
- **Декоратор `@abstractmethod`.** Используется для обозначения абстрактных методов — методов, которые должны быть реализованы в подклассах.

Назначение абстрактных классов и методов

- **Определение интерфейса.** Абстрактный класс задаёт набор методов, которые должны быть реализованы в его подклассах. Это позволяет гарантировать согласованность структуры и поведения.
- **Предотвращение инстанцирования.** Классы, определённые с использованием `abc` и декоратора `@abstractmethod`, не могут быть созданы напрямую — это защищает от использования неполных определений.
- **Общее поведение.** Абстрактные классы могут содержать как абстрактные, так и методы с реализацией, что позволяет включать общее поведение для всех подклассов.
- **Эмуляция интерфейсов.** В Python абстрактные классы без конкретных методов могут выступать в роли интерфейсов.

Использование

- **Проектирование фреймворков и библиотек.** Абстрактные классы помогают создать API, где пользователи должны наследоваться от базовых классов, реализуя ожидаемый интерфейс.
- **Большие проекты.** Позволяют строго контролировать архитектуру, снижать вероятность ошибок и облегчать понимание кода.
- **Командная разработка.** Задают «правила игры» для других разработчиков.
- **Системы с иерархией классов.** Подходят для ситуаций, когда существует естественная связь «is-a» между абстрактным классом и его наследниками.

Пример использования: создание системы управления транспортными средствами, где каждый тип транспорта должен уметь двигаться и останавливаться.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def move(self):
        pass
```

```
@abstractmethod
def stop(self):
    pass

class Car(Vehicle):
    def move(self):
        print("Car is moving")

    def stop(self):
        print("Car has stopped")
```

В этом примере `Vehicle` — абстрактный класс, а `Car` и `Bicycle` реализуют методы `move` и `stop` по-своему.

Дополнительные возможности

- **Абстрактные свойства.** Можно требовать реализацию не только методов, но и свойств с помощью комбинации `@property` и `@abstractmethod`
- **Сочетание с другими декораторами.** Вместе с `@abstractmethod` можно использовать и другие декораторы, включая `@staticmethod`, `@classmethod` и `@property`
- **Динамическое добавление абстрактных методов.** Динамически добавлять абстрактные методы в класс или модифицировать их статус после создания можно только с помощью функции `update_abstractmethods()`.

БИЛЕТ 38: Визуальное событийно-управляемое программирование. Основные концепции GUI (на примере библиотек Tkinter или PyQt).

Визуальное событийно-управляемое программирование в Python связано с созданием графических интерфейсов (GUI), где выполнение программы определяется событиями — действиями пользователя (клики, ввод текста, перемещение мыши и т. д.). Две популярные библиотеки для работы с GUI в Python — Tkinter и PyQt — используют разные подходы к обработке событий.

Событийно-ориентированное программирование (Event-Driven Programming) — парадигма, при которой выполнение программы определяется событиями. Программа «ожидает» наступления определённых событий и реагирует на них.

Виджеты — строительные блоки графического интерфейса (кнопки, текстовые поля, метки, меню и др.).

Цикл обработки событий (Event loop) — бесконечный цикл, который отслеживает события и вызывает обработчики.

Менеджеры компоновки (layout system) — механизмы, которые определяют, как виджеты размещаются относительно друг друга в окне приложения. Это позволяет создавать адаптивные интерфейсы, которые корректно масштабируются при изменении размеров окна.

Tkinter — стандартная библиотека Python, которая входит в состав языка. Она подходит для начинающих и простых проектов.

Система привязки (binding) — подход, при котором функция-обработчик привязывается к определённому событию виджета. Например, можно привязать функцию к событию нажатия левой кнопки мыши.

Пример привязки функции к событию нажатия кнопки:

```
def on_button_click():
```

```
    print("Кнопка была нажата!")
```

```
root = tk.Tk()
```

```
root.title("Tkinter События")
```

```
root.geometry("300x200")
```

```
button = tk.Button(root, text="Нажми меня")
```

```
button.pack(pady=50)
```

```
# Привязываем функцию к событию нажатия левой кнопки мыши
```

```
button.bind("<Button-1>", lambda event: on_button_click())
```

Особенности Tkinter:

- предлагает три менеджера компоновки: place, pack и grid;
- позволяет отслеживать различные события, включая клавиатурные нажатия, движение мыши и другие.

PyQt — библиотека, основанная на фреймворке Qt. Она позволяет создавать кроссплатформенные приложения с богатым интерфейсом.

Система сигналов и слотов — ключевой механизм взаимодействия между компонентами интерфейса. Сигналы — это события, генерируемые виджетами, а слоты — функции, которые вызываются при генерации сигналов.

Пример подключения сигнала к слоту:

```
from PyQt5.QtWidgets import QApplication, QPushButton, QVBoxLayout, QWidget

class MyApp(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()

    def initUI(self):
        layout = QVBoxLayout()
        self.button = QPushButton('Нажми меня', self)
        self.button.clicked.connect(self.on_button_click) # Подключаем сигнал к слоту
        layout.addWidget(self.button)
        self.setLayout(layout)
        self.setWindowTitle('PyQt События')
        self.setGeometry(300, 300, 300, 200)

    def on_button_click(self):
        print('Кнопка была нажата!')

if __name__ == '__main__':
    app = QApplication(sys.argv)
    ex = MyApp()
    ex.show()
    sys.exit(app.exec_())
```

Особенности PyQt:

- использует более структурированный подход с сигналами и слотами, что делает код более читаемым при сложной логике обработки событий;

- позволяет создавать пользовательские сигналы, что полезно при разработке сложных приложений с многокомпонентной архитектурой;
- предоставляет класс `QThread` для работы с многопоточными приложениями, что позволяет выполнять длительные операции в фоновом режиме, не блокируя интерфейс.

БИЛЕТ 39: Компоненты интерфейса (виджеты): кнопки, поля ввода, метки. Их свойства и события.

В Python для создания графического интерфейса (GUI) часто используется библиотека **Tkinter**. Она предоставляет набор виджетов — базовых элементов для построения интерфейса, с которыми взаимодействует пользователь. Среди них — кнопки, поля ввода и метки. У каждого из них есть свойства и они могут реагировать на события.

Кнопки (Button)

Button — виджет для создания кнопок, на которые пользователь может нажимать.

Некоторые свойства:

- **text** — текст, отображаемый на кнопке.
- **command** — функция, которая будет вызвана при нажатии на кнопку.
- **width** и **height** — ширина и высота кнопки.
- **bg** (background) и **fg** (foreground) — цвет фона и текста соответственно.

Пример использования:

```
def button_click():  
    print("Нажата кнопка")  
  
root = tk.Tk()  
root.title("Пример кнопок")  
button = tk.Button(root, text="Щелкните меня", command=button_click)  
button.pack()  
root.mainloop()
```

Поля ввода (Entry)

Entry — виджет для создания однострочного поля ввода.

Некоторые свойства:

- **width** — ширина поля в символах.

- **bd** (borderwidth) — ширина границы поля.

Пример:

```
ent = Entry(root, width=20, bd=3)
ent.pack()
```

Метки (Label)

Label — виджет для отображения текста или изображений. Текст в метке нельзя изменить пользователем, поэтому она часто используется для подписей, инструкций и информационных сообщений.

Некоторые свойства:

- **text** — текст, отображаемый на метке.
- **font** — шрифт текста.

Пример:

```
label = tk.Label(root, text="Это метка", font="Arial 14")
label.pack()
```

События и их обработка

События — действия или происшествия, которые происходят в системе управления графическим интерфейсом и на которые приложение может реагировать. В Tkinter для привязки обработчиков событий используется метод **bind()**.

Некоторые типы событий:

- клики мышью (например, <Button-1> — клик левой кнопкой мыши, <Button-3> — правый клик);
- нажатия клавиш (например, <KeyPress>, <KeyRelease>);
- движение мыши (например, <Motion>);
- получение/потеря фокуса (например, <FocusIn>, <FocusOut>).

Пример привязки события к кнопке:

```
def handle_click(event):
    print("Button clicked!")

window = tk.Tk()
button = tk.Button(window, text="Click Me")
button.bind("<Button-1>", handle_click)
button.pack()
```

Функция-обработчик события должна принимать в качестве параметра объект события (например, event).

Важно: в Tkinter события описываются с помощью зарезервированных ключевых слов, а их названия заключаются в угловые скобки.

Для открепления события от виджета используется метод **unbind()**, в который передаётся шаблон события.

БИЛЕТ 40: Разработка оконного приложения: создание главного окна, обработка событий (сигналов и слотов).

Разработка оконного приложения с использованием Tkinter и PyQt включает создание главного окна и обработку событий с помощью механизмов сигналов и слотов.

Tkinter — стандартная библиотека Python для создания графических пользовательских интерфейсов (GUI). Она не требует дополнительной установки и подходит для начинающих разработчиков.

Создание главного окна:

```
import tkinter as tk

root = tk.Tk() # Создание основного окна
root.title("Моё первое приложение") # Установка заголовка
root.geometry("800x600") # Установка размеров окна
root.minsize(400, 300) # Минимальный размер окна

root.mainloop() # Запуск цикла обработки событий
```

Метод `mainloop()` удерживает окно открытым и позволяет пользователю взаимодействовать с приложением.

Обработка событий: в Tkinter используется механизм привязки обработчиков событий (event binding). Для привязки события к определённому виджету применяется метод `bind()`. Например:

```
def on_key_press(event):
    print(f"Нажата клавиша: {event.keysym}") [9](https://victor-komlev.ru/programmirovaniye-graficheskogo-interfejsa-polzovatelya-s-pomoshhyu-tkinter-v-python/)

root.bind("<Key>", on_key_press)
```

PyQt — библиотека для создания GUI, основанная на концепции сигналов и слотов.

Создание главного окна: можно использовать

класс `QWidget` или `QMainWindow` (главное окно приложения с меню, панелями инструментов и строкой состояния).

Пример создания окна с помощью QWidget:

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget

app = QApplication(sys.argv)
w = QWidget()
w.resize(450, 350)
w.move(300, 300)
w.setWindowTitle('Окно простейшее')
w.show()

sys.exit(app.exec())
```

Обработка событий (сигналы и слоты):

- **Сигналы** — это уведомления, которые виджеты испускают при определённых событиях (нажатие кнопки, изменение текста и т. д.).
- **Слоты** — функции, которые вызываются в ответ на эти сигналы. Для подключения сигнала к слоту используется метод `connect()`

Пример подключения сигналов к слотам:

```
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton,
QLineEdit

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Обработка событий в PyQt")
        self.setGeometry(100, 100, 300, 200)

        self.button = QPushButton("Нажми меня", parent=self)
        self.entry = QLineEdit(parent=self)

        self.button.clicked.connect(self.button_click)

    def button_click(self):
        self.label.setText(f"Привет, {self.entry.text()}!")

app = QApplication(sys.argv)
window = MainWindow()
window.show()
sys.exit(app.exec_())
```

Дополнительные возможности:

- **Пользовательские сигналы** — можно создавать собственные сигналы для передачи данных между компонентами приложения.

- **Фильтры событий** — позволяют перехватывать и обрабатывать события до того, как они достигнут целевого объекта.
- **Мультипоточная обработка** — сигналы и слоты поддерживают безопасное взаимодействие между потоками.